Smoothing Functional Data with A Roughness Penalty

Pang Du

Department of Statistics
Virginia Tech

Why roughness penalties?

- Controlling smoothness by limiting the number of basis functions is discontinuous; roughness penalties allow continuous control over smoothness.
- We want to be able to define "smooth" in ways that are appropriate to our problems.
  - We may want a smooth derivative rather than just a smooth function.
  - What is smooth in one situation is not smooth in another. Smoothness has to be defined differently for periodic functions, for example.
- We find that roughness penalty smoothing gives better results.
- Roughness penalties are connected to fitting data by a differential equation; they are models for process dynamics.

Objectives

We have two competing objectives:

- Fit the data well; keep bias low.
- Keep the fit smooth so as to
  - filter out noise
  - get better estimates of derivatives

$$\text{Mean squared error} = \text{Bias}^2 + \text{Sampling Variance}.$$

We can often greatly reduce MSE by trading a little bias off against a lot of sampling variance.

Quantifying roughness

- The classic: **curvature in the function**

$$\mathrm{PEN}_2(x) = \int [D^2 x(s)]^2 ds.$$

$[D^2 x(s)]^2$ measures the squared curvature in x at s. This penalty measures *total squared curvature*.

- **Curvature in acceleration**:

$$\mathrm{PEN}_4(x) = \int [D^4 x(s)]^2 ds.$$

- These two penalties also define what we mean by "smooth"; any function that has zero penalty is "hyper-smooth". A straight line for the classic, a cubic polynomial for the acceleration penalty.

Harmonic acceleration

- If the process is periodic, it is natural to think of a constant + sinusoid as "hyper-smooth". This suggests that we use

$$PEN_H(x) = \int [D^3x(s) + \omega^2 Dx(s)]^2 ds,$$

where $2\pi/\omega$ is the period.

- The functions $1, \sin(\omega t)$, and $\cos(\omega t)$ all have zero penalties, as does any linear combination of them.

Some questions to think about

- Writing $Lx(s) = D^3x(s) + \omega^2 Dx(s)$, we have

$$PEN_H(x) = \int [Lx(s)]^2 ds.$$

- Can we think of other differential operators L that might be useful?
- If we have a small number of "hyper-smooth" functions in mind, can we find a differential operator L that will assign zero penalty to them?
- Can we use the data themselves to tell us something about the right differential operator L?

Notation

- Notation:
  - $\mathbf{y}$ is the n-vector of data y_j to be smoothed.
  - $\mathbf{t}$ is the n-vector of values of t_j.
  - $\mathbf{W}$ is a symmetric positive definite weight matrix, allowing for possible covariance structure among residuals.
  - $x(\mathbf{t})$ is the n-vector of fitted values, and $x(\mathbf{t})$ has the basis function expansion

$$x(\mathbf{t}) = \sum_{k=1}^{K} c_k \phi_k(\mathbf{t}) = \mathbf{c}'\boldsymbol{\phi}(\mathbf{t}).$$

- The penalized least squares criterion is

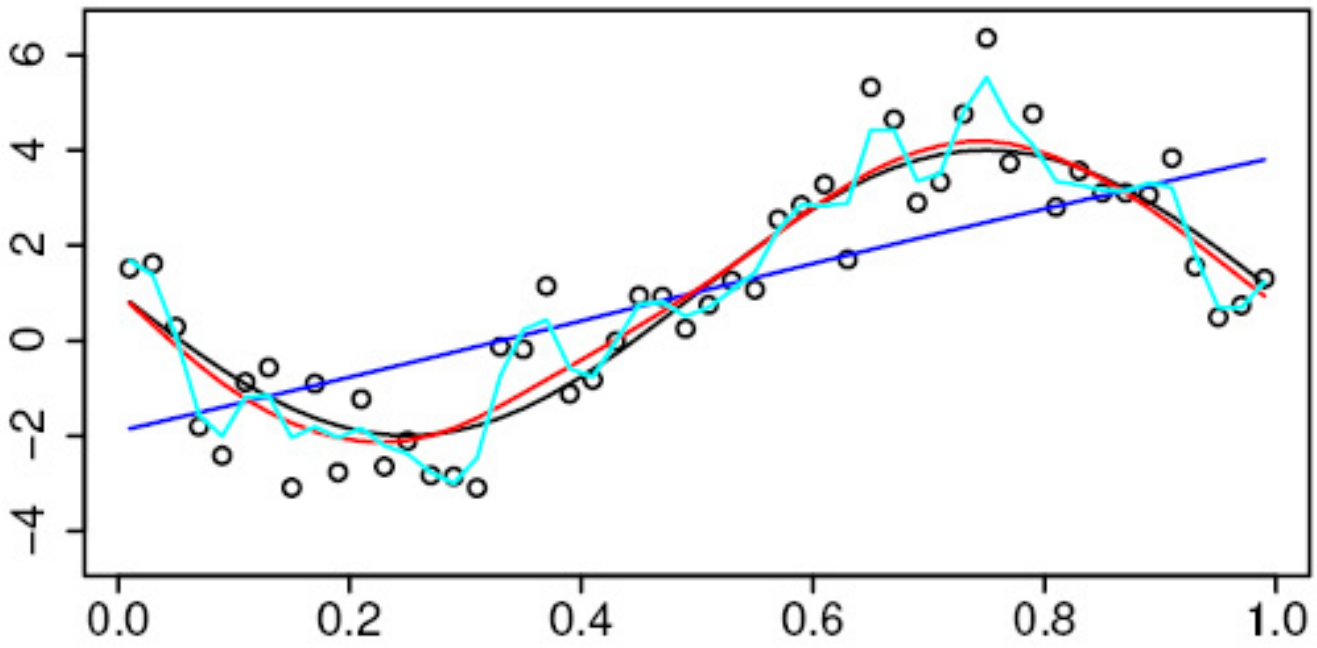
$$\mathrm{PENSSE}_\lambda(x|\mathbf{y}) = [\mathbf{y} - x(\mathbf{t})]'\mathbf{W}[\mathbf{y} - x(\mathbf{t})] + \lambda \mathrm{PEN}(x)$$

How the smoothing parameter works?

Smoothing parameter λ controls the amount of roughness.

- As $\lambda \to 0$, roughness matters less and less, and $x(\mathbf{t})$ fits the data better and better.
- As $\lambda \to \infty$, roughness matters more and more, and $x(\mathbf{t})$ becomes more and more "hyper-smooth".
- Our job is to find the right value where we trade enough bias off against sampling variance to minimize mean squared error.

Effect of smoothing parameter



Three estimates of x are calculated by minimizing $\sum_{i=1}^{n}(y_i - x(t_i))^2 + n\lambda \int_0^1 [x''(t)]^2 dt$ at $\log_{10} n\lambda = 0, -3, -6$.

The Representer Theorem

Theorem

Let $\hat{x}$ be the minimizer of the penalized least squares

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - x(t_i))^2 + \lambda \int_0^1 [x^{(m)}(t)]^2 dt$$

in the space $\mathcal{H} = \{x : x^{(m)}$ is square-integrable$\}$ equipped with the inner product $\langle x_1, x_2\rangle = \sum_{\nu=0}^{m-1} \int_0^1 x_1^{(\nu)}(t) \int_0^1 x_2^{(\nu)}(t)dt + \int_0^1 x_1^{(m)} x_2^{(m)} dt$. Then

$$\hat{x}(t) = \sum_{\nu=0}^{m-1} d_\nu \varphi_\nu(t) + \sum_{i=1}^{n} b_i R_1(t_i, t),$$

for some coefficient vectors $\mathbf{d} = (d_0, \ldots, d_{m-1})'$ and $\mathbf{b} = (b_1, \ldots, b_n)'$.

The Representer Theorem (cont'd)

- Decompose $\mathcal{H} = \mathcal{H}_0 \oplus \mathcal{H}_1$.
- $\mathcal{H}_0$: the m-dimensional null space of the penalty that contains all the lower order polynomials and has a basis $\{\varphi_0, \ldots, \varphi_\nu\}$.
- $\mathcal{H}_1$: complement of $\mathcal{H}_0$, which contains all the high-order functions and possesses a *reproducing kernel* R_1.
- Back to the notation in the early expression of x, we have $K = m + n$, $\mathbf{c}' = (\mathbf{d}', \mathbf{b}')$ and $\boldsymbol{\phi} = (\varphi_0, \ldots, \varphi_{m-1}, R_1(t_1, \cdot), \ldots, R_1(t_n, \cdot))'$.

Generalization of the Representer Theorem

The theorem can be extended to

- any other linear differential operators besides D^m: e.g., the harmonic differential operator.
- any other loss functions besides the MSE that depend on x only through the $x(t_i)$'s: e.g., negative log likelihood of the generalized regression models for data from exponential family distributions.

The roughness penalty matrix

- For the classic penalty,

$$\begin{aligned} \mathrm{PEN}_2(x) &= \int [D^2 \mathbf{c}' \phi(t)]^2 dt \\ &= \mathbf{c}' \int [D^2 \phi(t)][D^2 \phi(t)]' dt \mathbf{c} \\ &= \mathbf{c}' \mathbf{R} \mathbf{c}. \end{aligned}$$

- The order-m roughness penalty matrix $\mathbf{R}$ is

$$\mathbf{R} = \int [D^m \phi(t)][D^m \phi(t)]' dt = \int (D^m \phi)(D^m \phi)'.$$

- substitute L for D^m for more general roughness penalties.

The estimates for $\mathbf{c}$ and $\mathbf{y}$

- Φ is the $n \times K$ matrix of basis function values $\phi_k(t_j)$.
- The penalized least squares criterion becomes

$$\text{PENSSE}(\mathbf{y}|\mathbf{c}) = (\mathbf{y} - \Phi\mathbf{c})'\mathbf{W}(\mathbf{y} - \Phi\mathbf{c}) + \lambda\mathbf{c}'\mathbf{R}\mathbf{c}.$$

- This is quadratic in $\mathbf{c}$, and is minimized by

$$\hat{\mathbf{c}} = (\Phi'\mathbf{W}\Phi + \lambda\mathbf{R})^{-1}\Phi'\mathbf{W}\mathbf{y}.$$

The smoothing matrix $\mathbf{S}_{\phi,\lambda}$

- The data-fitting vector $\hat{\mathbf{y}} = x(\mathbf{t})$ is

$$\hat{\mathbf{y}} = \Phi(\Phi'\mathbf{W}\Phi + \lambda\mathbf{R})^{-1}\Phi'\mathbf{W}\mathbf{y}.$$

- Smoothing matrix

$$\mathbf{S}_{\phi,\lambda} = \Phi(\Phi'\mathbf{W}\Phi + \lambda\mathbf{R})^{-1}\Phi'\mathbf{W}$$

maps the data into the fit, and has many useful applications.

Equivalent degrees of freedom

- It is useful to compare a fit using a roughness penalty to one using a fixed number of basis functions.
- A measure of the "degrees of freedom" in a roughness penalized fit is

$$df(\lambda) = \text{trace}(\mathbf{S}_{\phi,\lambda}).$$

- This corresponds to the number of basis functions K in an un-penalized fit.

Cross-validation

If we choose smoothing parameter λ by cross-validation, we

- set aside a subset of data, the **validation sample**
- call the remaining data the **training sample**
- fit the model to the training sample
- assess fit to the validation sample
- repeat these steps a number of times (often five or ten times), and choose the λ value that gives the best fit averaged across the repetitions.

Delete-one cross-validation

We can also, for a sequence of values of λ,

- set aside each observation (t_j, y_j) in turn
- fit the data with the rest of the sample,
- sum fits to the left out values to get a *cross-validated error sum of squares* $\mathrm{CV}(\lambda)$.
- select the λ value that minimizes $\mathrm{CV}(\lambda)$.

Generalized cross-validation

- Cross-validation is time-consuming, and tends too often to under-smooth the data.
- The generalized cross-validation criterion is

$$\mathrm{GCV}(\lambda) = \left(\frac{n}{n - df(\lambda)}\right)\left(\frac{\mathrm{SSE}}{n - df(\lambda)}\right),$$

  where df is the equivalent degrees of freedom of the smoothing operator.
- $\mathrm{GCV}(\lambda)$ approximates $\mathrm{CV}(\lambda)$.
- The right factor is just the unbiassed estimate σ_e^2 of residual variance familiar in regression analysis.
- The left factor further "discounts" this measure to allow for the influence of optimizing with respect to λ.

Simulation: Jolicoeur's growth model

- How does GCV work in a simulated data example?
- A parametric growth model by Pierre Jolicoeur at the Université de Montréal (Canada) offers a nice test problem.
- The growth curve in Jolicoeur (1992) has the form

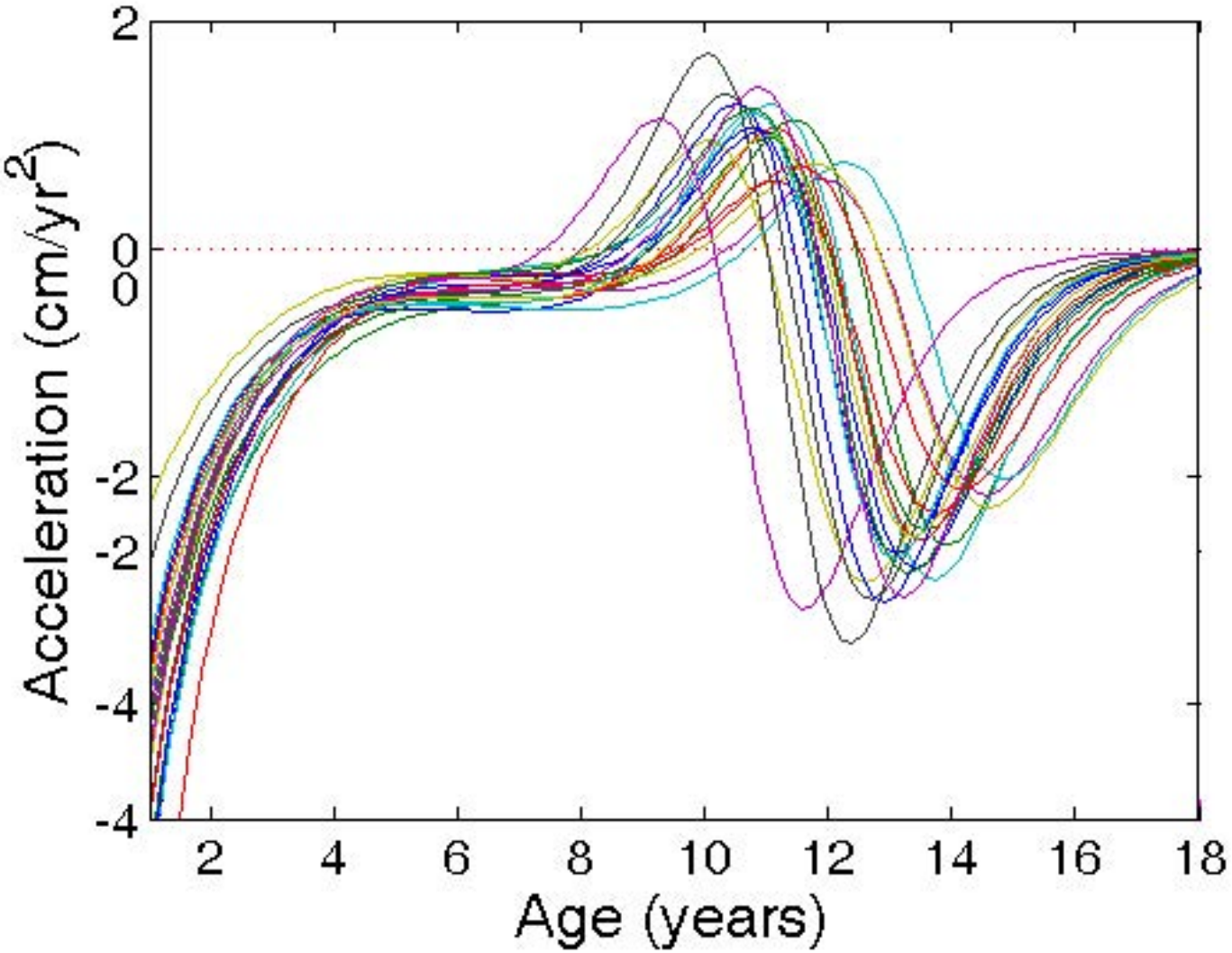
$$h(t) = \frac{a \sum_{k=1}^{3} [b_k(t+e)]^{c_k}}{1 + \sum_{k=1}^{3} [b_k(t+e)]^{c_k}}.$$

- How well do we estimate the Jolicoeur acceleration curves?

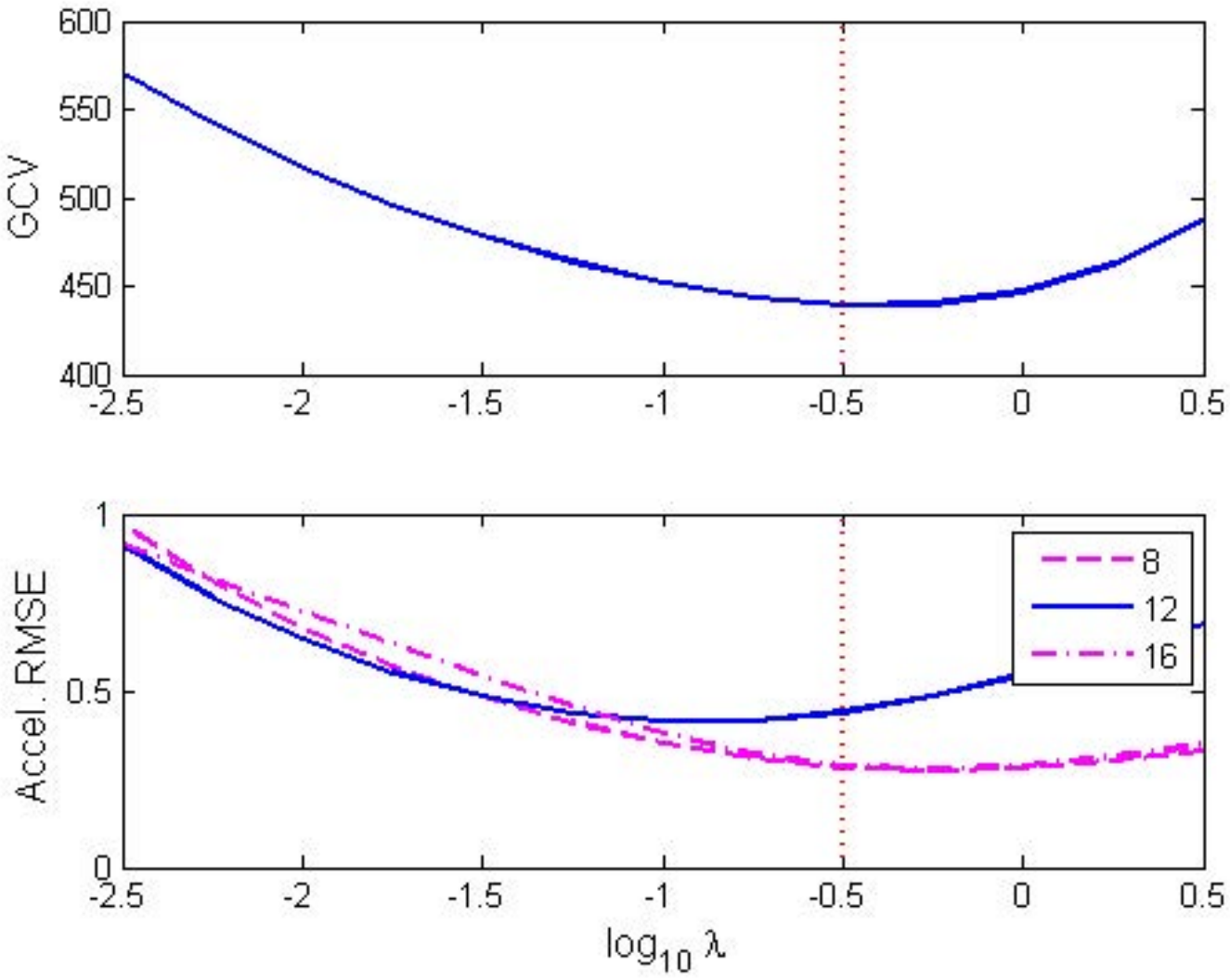
Simulation: Setup

- Parameter fits to the Fels growth data (Roche, 1992) by Bock (2000): $a = 164.7, e = 1.474, \mathbf{b} = (0.3071, 0.1106, 0.0816)', \mathbf{c} = (3.683, 16.665, 1.474)'$.
- Standard error curve $s(t)$ is also available in the textbook, and used to simulate normal random errors that are added to the smooth growth curve.
- Sampling ages t_i: quarterly for 1-2 years, annually for 2-8 years, and every other year after that to 18 years of age.
- $1/s^2(t_i)$ define the entries of the diagonal weight matrix $\mathbf{W}$.
- We simulate 1000 samples.
- We smooth using a range of values of λ, and note the value giving the best value of GCV.

20 Jolicoeur acceleration curves



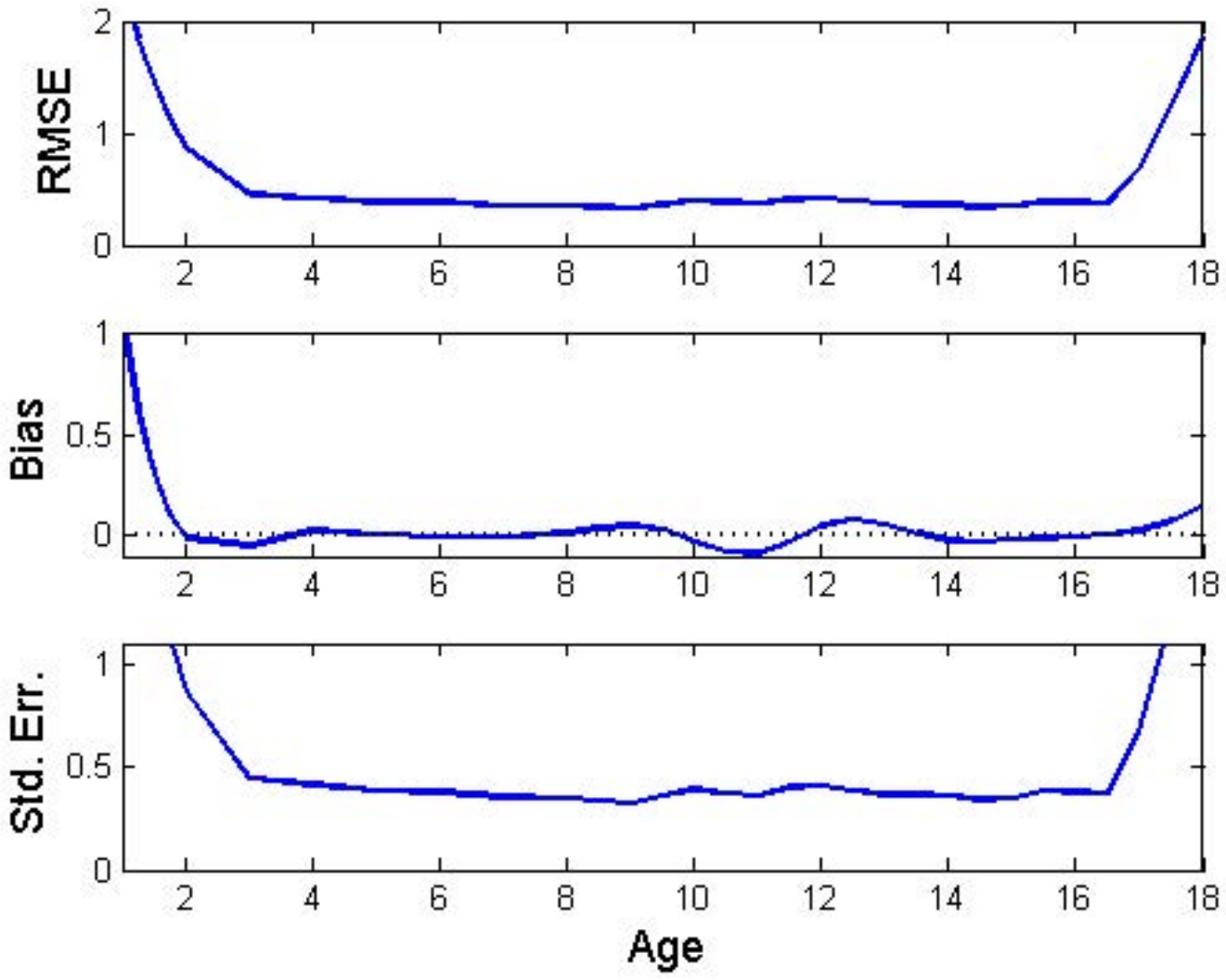
GCV and RMSE


600
550
500
450
400
GCV
-2.5
-2
-1.5
-1
-0.5
0
0.5
1
0.5
0
Accel. RMSE
8
12
16
log10 λ

What we see?

- In the top panel, GCV favors $\lambda = 10^{-0.5} \approx 0.316$.
- This is about right for optimal RMSE for ages 8 and 16, but less smoothing would be better for age 12, in the middle of the pubertal growth spurt.
- One smoothing parameter value does not work best for all ages, but
- The value chosen by GCV certainly does a fine job.

RMSE, bias, and SE



What we see?

- The performance of the smoothing spline estimate deteriorates badly at the extremes.
- The sharp curvature at the pubertal growth spurt also causes some problems.
- Except at the extremes and PGS, the bias is negligible. The standard error is about the same as RMSE.
- Would we do better at the extremes if the smooth respected monotonicity?

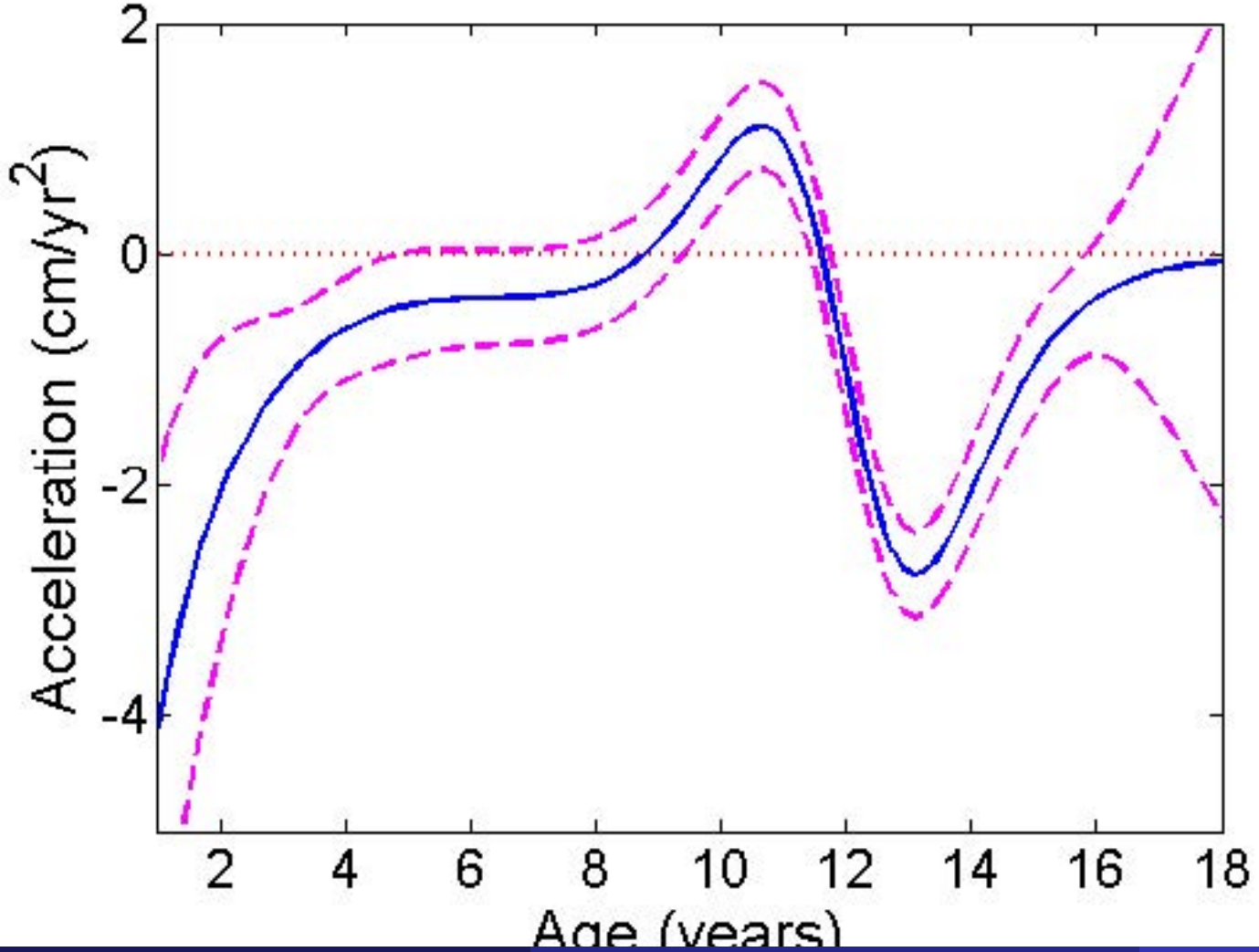
Point-wise CIs

- *Linear* mapping from data $\mathbf{y}$ to the coefficient vector $\mathbf{c}$: $\mathbf{c} = \mathbf{S}\mathbf{y}$.
- Straightforward to work out the standard error of any linear functional of a curve defined by $\mathbf{c}$.
- The variance of a quantity $\rho(x) = \mathbf{M}\mathbf{c}$ associated with linear mapping $\mathbf{M}$, or explicitly $\mathbf{M}(x)$, is

$$\mathrm{Var}[\rho(\hat{x})] = \mathbf{M}\mathbf{S}\Sigma_e\mathbf{S}'\mathbf{M}',$$

  where the smoothing matrix $\mathbf{S}_{\phi,\lambda} = \Phi\mathbf{S}$.
- Also need a good estimate of the covariance matrix Σ_e of the residual vector.
- Special case: $\rho(x) = x(t_i)$ yields the CI of $x(\cdot)$ at $t = t_i$.

95% point-wise CIs for growth acceleration



A scalable approximation

Putting a knot at each observation can have heavy computational burden when n is large. A scalable approximation proposed in Kim and Gu (2004, JRSSB):

- randomly select an order of $n^{2/9}$ many knots from all the observations,
- achieve the same asymptotic convergence rate.
- practical suggestion on the number of knots: $10n^{2/9}$.

Roughness penalty approach in the textbook

The roughness penalty approach in the textbook uses the B-spline basis functions instead of basis functions used in smoothing splines.

- Representer theorem does not apply: estimate is an approximate solution.
- Such approximation works okay in practice once enough basis functions are included.
- This is also the roughness penalty approach provided in the R package `fda` and the one we will use in this class.

Summary

- Roughness penalization, also called *regularization*, is a flexible and effective way to ensure that an estimated function is "smooth".
- We can tailor the definition of "smooth" to our needs.
- The roughness penalty idea extends to any type of *functional parameter* that we want to estimate from the data.
- Roughness penalties are one of the main ways in which we exploit the smoothness that we assume in the process generating the data.

Roughness and energy

- "Roughness" is like *energy* in physics
- Roughness requires energy to produce, and smoothness implies limited energy.
- Where we imagine that the amount of energy behind the data is limited, it is natural to assume smoothness.